DOCUMENTACIÓN TÉCNICA

Proyecto Final — Automatización con n8n y LLMs

Bruno Ignacio Solla – Comisión 97150

Ford Repuestos

Sistema de Triage Inteligente de Mensajes

Campo	Detalle
Institución	Coderhouse
Curso	IA Automation
Fecha	Marzo 2026
Versión	1.0 — Final
Stack	n8n Cloud + Groq (Llama-3.3-70b-versatile) + Telegram + Discord + Google Sheets

1. Resumen Ejecutivo

Ford Repuestos opera su tienda en Mercado Libre y atiende clientes por Telegram. El equipo enfrenta un alto volumen de mensajes diarios que incluyen reclamos, consultas de preventa y seguimiento de pedidos.

Este proyecto implementa un sistema automático de clasificación y respuesta utilizando n8n como orquestador, Groq (Llama-3.3-70b-versatile) como motor de IA, Telegram como canal cliente, Discord para notificaciones internas y Google Sheets para logs y auditoría.

> OBJETIVO: Reducir el tiempo de primera respuesta de horas a segundos, clasificar automáticamente cada mensaje y generar respuestas personalizadas según el tipo de consulta.

2. Arquitectura del Sistema

2.1 Nodos del Flujo Principal

Nodo	Tipo	Función
Telegram Trigger	Trigger	Detecta mensajes nuevos en el bot de Telegram
Preparar Datos del Mensaje	Code JS	Extrae chat_id, nombre, username, mensaje. Filtra vacíos, emojis, puntuación y mensajes < 3 caracteres
¿Mensaje válido?	IF	Bifurca según si el mensaje pasó la validación
Telegram — Mensaje Vacío	Telegram	Avisa al cliente que no se pudo leer su mensaje
LLM #1 — Clasificador	Basic LLM Chain	Clasifica en: reclamo / consulta_preventa / seguimiento
Groq — Llama3 Clasificador	Groq API	Modelo Llama-3.3-70b-versatile, temperature 0.1 para máxima consistencia
Parsear Clasificación	Code JS	Extrae JSON del LLM y fusiona con datos del cliente
Switch — Tipo de Mensaje	Switch	Bifurca el flujo según la categoría (4 ramas)
LLM #2A — Reclamo	Basic LLM Chain	Respuesta empática + acción interna para el equipo
LLM #2B — Preventa	Basic LLM Chain	Identifica tipo de consulta, solicita datos del vehículo
LLM #2C — Seguimiento	Basic LLM Chain	Acuse de recibo con tiempo de respuesta estimado
Parsear Respuesta (x3)	Code JS	Extrae JSON de cada LLM con fallback de emergencia
Telegram — Responder Cliente (x3)	Telegram	Envía respuesta generada al chat del cliente
Sheets — Log (x3)	Google Sheets	Registra el mensaje procesado con todos sus datos
Discord (x3)	Discord Webhook	Notifica al equipo en el canal correspondiente
Sheets — Log Fallback	Google Sheets	Registra mensajes no clasificados para revisión manual

Nodo	Tipo	Función
Telegram — Fallback Cliente	Telegram	Respuesta genérica para mensajes no clasificados
Discord — Fallback	Discord Webhook	Alerta de mensaje no clasificado al equipo

2.2 Rama de Error Técnico

Nodo	Función
Error Trigger	Se activa cuando cualquier nodo del workflow falla
Parsear Error Técnico	Extrae nodo que falló, tipo de error, severidad y stack trace
Sheet — Error Técnico	Registra el error completo en hoja Errores_Tecnicos
Notificacion Error Técnico	Alerta en Discord #errores-tecnicos con severidad y execution ID

3. Configuración Técnica

3.1 Credenciales

Servicio	Tipo	Dónde obtenerla
Telegram	API Token del Bot	@BotFather en Telegram → /newbot
Groq	API Key	console.groq.com → API Keys → Create
Google Sheets	OAuth2	Autenticación con cuenta Google en n8n
Discord (x5)	Webhook URL por canal	Canal → Editar → Integraciones → Webhooks

3.2 Google Sheets — 3 Hojas

Hoja: Logs

fecha · canal · nombre_cliente · username_telegram · chat_id · mensaje_original · categoria · urgencia · resumen_ia · respuesta_enviada · accion_interna · estado · happy_path

Hoja: Fallback

fecha · canal · nombre_cliente · username_telegram · chat_id · mensaje_original · categoria · resumen_ia · respuesta_llm_cruda · estado · happy_path

Hoja: Errores_Tecnicos

fecha · workflow · execution_id · nodo_que_fallo · tipo_error · mensaje_error · severidad · stack_trace · resuelto

3.3 Canales de Discord

Canal	Rama	Contenido
#reclamos	Rama Reclamo	Urgencia, cliente, mensaje, resumen IA, respuesta enviada, acción requerida
#preventa	Rama Preventa	Tipo de consulta, chasis proporcionado, datos faltantes, respuesta enviada
#seguimiento	Rama Seguimiento	Número de pedido, respuesta enviada, recordatorio 2hs hábiles
#fallback	Rama Fallback	Mensaje original, respuesta cruda del LLM, alerta de revisión manual
#errores-tecnicos	Error Trigger	Nodo que falló, tipo, severidad, fecha, execution ID

4. Prompts de los Nodos LLM

4.1 LLM #1 — Clasificador

Parámetro	Valor
Modelo	Llama-3.3-70b-versatile
Temperature	0.1 — máxima consistencia para no romper el Switch
Max Tokens	150 — solo necesita devolver un JSON corto

Temperatura 0.1: garantiza que el LLM siempre devuelva el mismo formato JSON. Una temperatura alta podría generar variaciones que rompen el nodo Switch.

Categorías: reclamo · consulta_preventa · seguimiento

Salida JSON: { categoria, urgencia (alta/media/baja), resumen }

4.2 LLM #2A — Respuesta Reclamo

Parámetro	Valor
Temperature	0.5 — creatividad moderada para sonar humano
Max Tokens	400
Tono	Argentino, vos, plural (nosotros), máx. 4 oraciones
Adaptación	Disculpa si molesto, saludo si neutro

4.3 LLM #2B — Respuesta Preventa

Parámetro	Valor
Temperature	0.5
Max Tokens	400
Identificación	Distingue compatibilidad / disponibilidad / precio / mixta
Dato clave	Solicita número de chasis (VIN, 17 dígitos) siempre

4.4 LLM #2C — Respuesta Seguimiento

Parámetro	Valor
Temperature	0.4 — más formal que los anteriores
Max Tokens	300 — respuesta más corta
Dato clave	Detecta número de pedido ML (10-12 dígitos)

En las 3 ramas de categoría (reclamo, consulta_preventa y seguimiento) se utilizó el mismo motor para el LLM: Llama-3.3-70b-versatile

5. Robustez y Manejo de Errores

5.1 Validación de Entrada

El nodo Preparar Datos del Mensaje rechaza automáticamente:

- Mensajes vacíos o solo espacios
- Mensajes de menos de 3 caracteres reales
- Mensajes compuestos solo de emojis
- Mensajes compuestos solo de puntuación

5.2 Try/Catch en Parseos

Los tres nodos de parseo (Reclamo, Preventa, Seguimiento) tienen bloques try/catch. Si el LLM devuelve texto malformado o con markdown, el catch usa un objeto de fallback predefinido para no romper el flujo.

5.3 Rama Fallback

Si el Switch no reconoce la categoría devuelta por el clasificador, deriva a la rama fallback que registra el mensaje en la hoja Fallback de Sheets, envía respuesta genérica al cliente y alerta al equipo en Discord #fallback.

5.4 Severidad Automática en Error Trigger

Nodo que falla	Severidad
LLM / Groq	ALTA
Telegram	ALTA
Google Sheets	MEDIA
Error Trigger	CRÍTICA
Otros	MEDIA

6. Casos de Prueba Realizados

#	Mensaje enviado	Clasificación obtenida	Resultado
1	Me llegó la pieza completamente rota, no puedo creer que manden así los productos	reclamo · alta	✅Correcto
2	Hola! tienen filtro de aceite para un Ford Focus 2015?	consulta_preventa · media	✅Correcto
3	hola buenas (forzado a fallback)	fallback	✅Correcto

Todas las pruebas generaron respuesta en Telegram, notificación en Discord y registro en Google Sheets correctamente.

7. Limitaciones Conocidas

- El LLM puede clasificar incorrectamente mensajes muy ambiguos sin contexto suficiente
- Mensajes en idiomas distintos al español pueden reducir la precisión
- El bot no procesa imágenes, audios ni archivos adjuntos — solo texto plano
- Groq tiene límites de rate en el plan gratuito — producción con alto volumen requeriría plan pago
- Los números de pedido ML son detectados por el LLM sin validación contra la base de datos real

8. Stack Tecnológico

Componente	Tecnología	Detalle
Orquestación	n8n Cloud	Starter Plan
LLM	Groq — Llama-3.3-70b-versatile	4 instancias: Clasificador, Reclamo, Preventa, Seguimiento
Trigger / Canal cliente	Telegram Bot API	Bot creado con @BotFather
Notificaciones equipo	Discord Webhooks	5 canales: reclamos, preventa, seguimiento, fallback, errores-tecnicos
Logs y auditoría	Google Sheets	3 hojas: Logs, Fallback, Errores_Tecnicos
Código custom	JavaScript (n8n Code nodes)	Preparar datos, parsear respuestas, parsear error